# Web Application Security Testing on DVWA

Cyber Security Incident Response Analyst

## Supervised by:

**Eng: Ali Khafagy**

---

## Program:

**Digital Egypt Pioneers Initiative (DEPI)**

---

## Submitted by:


**- Ahmed Ehab Fathi Mohamed**

**- Mazen Ahmed Ahmed**

**- Mariam Sami Said**

**- Basma mohamed salama**

**- Yasser Hamdy Mohamed**

**- Manar Hany ezzat**

# Introduction:

**The Damn Vulnerable Web Application (DVWA) is a deliberately vulnerable web application designed for security professionals and students to practice and understand common web application vulnerabilities in a controlled environment. It provides a safe platform to simulate real-world attacks and analyze their impact without affecting actual production systems.**

**In today's digital world, web applications are one of the most targeted components by attackers. Vulnerabilities such as SQL Injection, Cross-Site Scripting (XSS), Command Injection, and others can lead to serious consequences including data breaches, unauthorized access, and system compromise. Therefore, understanding how these vulnerabilities work is essential for both offensive (penetration testing) and defensive (secure development) perspectives.**

**The primary objective of this project is to perform a comprehensive security assessment on DVWA by exploiting multiple vulnerabilities available within the application. Each vulnerability is analyzed in detail by demonstrating how the attack works, the steps required to exploit it, the expected results, and the potential risks associated with it.**

**Furthermore, this documentation also focuses on mitigation techniques and best practices that can be applied to prevent such vulnerabilities in real-world applications. This helps bridge the gap between theoretical knowledge and practical implementation of secure coding practices.**

---

# Objectives of the Project:

- **To understand and analyze common web application vulnerabilities.**
- **To perform practical exploitation of vulnerabilities using DVWA.**
- **To study how attackers manipulate user inputs and system behavior.**
- **To demonstrate the impact of each vulnerability.**
- **To identify and apply appropriate mitigation techniques.**
- **To gain hands-on experience in web application security testing.**

---

# Scope of the Project:

**This project focuses on testing and exploiting vulnerabilities provided within DVWA in a controlled environment. The scope includes:**

- **Performing attacks on different security levels (Low, Medium, and High where applicable).**
- **Analyzing the following vulnerabilities:**
    - **Brute Force**
    - **Command Injection**
    - **Cross-Site Request Forgery (CSRF)**
    - **File Inclusion (Local File Inclusion - LFI)**
    - **File Upload**
    - **Insecure CAPTCHA**
    - **SQL Injection**
    - **SQL Injection (Blind)**
    - **Weak Session IDs**
    - **Cross-Site Scripting (XSS - DOM-based , reflected , Stored)**
    - **Content Security Policy (CSP) Bypass**
    - **JavaScript Vulnerabilities**
    - **Authorization Bypass**
    - **Open HTTP Redirect**
    - **Cryptography Issues**
    - **API Vulnerabilities**
- **Documenting each vulnerability with:**
    - **Description**
    - **Exploitation steps**
    - **Payloads used**
    - **Results obtained**
    - **Mitigation techniques**

# Methodology:

**The methodology followed in this project is based on standard penetration testing phases:**

### 1. Reconnaissance

**Understanding the structure of the application, available inputs, and functionalities.**

### 2. Vulnerability Identification

**Identifying weak points in the application such as input fields, parameters, and authentication mechanisms.**

### 3. Exploitation

**Executing attacks using crafted payloads to exploit identified vulnerabilities.**

### 4. Analysis

**Observing the system response and analyzing the impact of each attack.**

### 5. Mitigation

**Providing recommendations and techniques to secure the application against such attacks.**

---

# Tools Used:

**The following tools were used during this project:**

- **Damn Vulnerable Web Application**
- **Burp Suite**
- **Web Browser (Google Chrome / Firefox)**
- **Virtual Machine Software (VirtualBox / VMware)**

---

# Vulnerability Analysis:

## 1. Brute Force Attack

### - Description:

**A brute force attack is a method used to gain unauthorized access by systematically trying all possible username and password combinations until the correct credentials are found.**

### - How the Attack Works:

**The attacker automates repeated login attempts using tools or scripts. In DVWA, the login form does not properly limit the number of attempts, making it vulnerable.**

### - Steps:

1. **Open the login page.**
2. **Intercept the request using Burp Suite.**
3. **Send the request to Intruder.**
4. **Use a wordlist for usernames and passwords.**
5. **Start the attack.**

### - Result:

**Successful login without knowing valid credentials.**

### - Mitigation:

- **Implement account lockout after failed attempts**
- **Use CAPTCHA**
- **Apply rate limiting**
- **Enable multi-factor authentication (MFA)**

# 2. Command Injection

## - Description:

**Command Injection is a vulnerability where user input is passed to system commands without proper validation, allowing attackers to execute arbitrary OS commands on the server and potentially gain unauthorized access or control.**

## - How the attack works:

**The attack works when user input is unsafely passed into system commands, allowing an attacker to inject additional commands using special characters (like &&, ;, or |) that get executed by the server.**

## - Steps:

1. **Locate input field (ping functionality).**
2. **Inject command:**

```
127.0.0.1 && whoami
```

## - Result:

**1. The injected command is executed on the server.**

**2. The output reveals sensitive system information (current user privileges).**

**3. This confirms successful remote command execution on the host system.**

## - Mitigation:

- **Validate and sanitize inputs**
- **Avoid system-level calls**
- **Use secure APIs**
- **Apply least privilege principle**

# 3. CSRF (Cross-Site Request Forgery)

## - Description:

**CSRF is a vulnerability that tricks authenticated users into unknowingly performing unintended actions on a web application.**

## - How the attack works:

**The attacker forces a logged-in user to send a malicious request that the server trusts because it includes the user's active session.**

## - Steps:

**1. Capture a valid request (password change).**

**2. Replicate it in a malicious HTML page.**

**3. Send the link to victim.**

## - Result:

**Unauthorized actions are executed using the victim's authenticated session.**

## - Mitigation:

- **Use CSRF tokens**
- **Validate HTTP referrer/origin**
- **Enable SameSite cookies**

# 4. File Inclusion (LFI)

## - Description:

**LFI is a vulnerability that allows attackers to include and read local files from the server through insecure file handling in web applications.**

## - How the attack works:

**The attacker manipulates file input parameters to load unintended system files using path traversal (../../etc/passwd).**

## - Steps:

1. **Identify a file inclusion parameter (?page=).**
2. **Modify the parameter to include path traversal payloads.**
3. **Example payload:**

```
../../etc/passwd
```

4. **The application processes the input without proper validation.**
5. **The server returns the contents of the targeted file.**

## - Result:

**Exposure of sensitive system files and confidential server information.**

## - Mitigation:

- **Use a strict whitelist for allowed files**
- **Disable dynamic file inclusion**
- **Apply strong input validation and filtering**
- **Prevent directory traversal patterns (../)**

# 5. File Upload

## - Description:

**File Upload is a vulnerability that allows attackers to upload malicious files (web shells) which may lead to server compromise.**

## - How it works:

**The server accepts a malicious file without proper validation, stores it in a public directory, and when the file is accessed through the browser, it gets executed as server-side code leading to remote code execution.**

## - Steps:

1. **Attacker uploads a malicious file (.php web shell).**
2. **The server accepts the file without proper validation.**
3. **The attacker accesses the uploaded file via browser.**
4. **The server executes the malicious code.**

## - Result:

**Remote Code Execution (RCE) on the server.**

## - Mitigation:

- **Restrict allowed file types (whitelist only safe extensions)**
- **Rename uploaded files to prevent execution**
- **Store uploaded files outside the web root**
- **Validate file content, not just extension**

# 6. Insecure CAPTCHA

## - Description:

**Insecure CAPTCHA occurs when CAPTCHA protection is weak or improperly implemented, allowing attackers to bypass it and perform automated requests.**

## - How it works:

**The application fails to properly validate CAPTCHA on the server side, allowing attackers to bypass or reuse requests without solving the challenge.**

## - Steps:

1. **Identify a form protected with CAPTCHA (login or password reset).**
2. **Submit a valid request and capture it using a proxy tool.**
3. **Modify or remove the CAPTCHA verification parameter.**
4. **Replay the request multiple times automatically.**
5. **The server processes the request without enforcing CAPTCHA validation.**

## - Result:

**CAPTCHA is bypassed, enabling automated attacks such as brute force and spam.**

## - Mitigation:

- **Enforce strong server-side CAPTCHA validation**
- **Use modern CAPTCHA systems like reCAPTCHA**
- **Implement rate limiting per IP/user**
- **Monitor and block repeated automated requests**

# 7. SQL Injection

## - Description:

**SQL Injection is a vulnerability that allows attackers to manipulate database queries by injecting malicious SQL code through user input.**

## - How it works:

**The application directly includes user input in SQL queries without proper sanitization, allowing attackers to modify the query logic.**

## - Steps:

1. **Identify an input field (login form).**
2. **Enter a malicious payload such as:**

```
' OR '1'='1
```

3. **The input is concatenated into the SQL query.**
4. **The database executes the modified query.**

## - Result:

- **Login bypass**
- **Unauthorized data access or extraction**

## - Mitigation:

- **Use prepared statements**
- **Use parameterized queries**
- **Validate and sanitize user input**
- **Apply least privilege for database accounts**

# 8. Blind SQL Injection

## - Description:

**Blind SQL Injection is a type of SQL injection where the attacker cannot see direct database responses but can infer information using true/false conditions or behavioral differences.**

## - How it works:

**The attacker injects conditional SQL queries and observes the application's response (page behavior, delay, or error differences) to extract data indirectly.**

## - Steps:

1. **Identify an injectable parameter.**
2. **Inject a boolean-based payload (true/false condition).**
3. **Observe differences in application response.**
4. **Use repeated conditions to extract database data bit by bit.**

## - Result:

**Slow but effective extraction of sensitive database information without direct output.**

## - Mitigation:

- **Use prepared statements**
- **Use parameterized queries**
- **Validate and sanitize all inputs**
- **Apply least privilege on database accounts**

# 9. Weak Session IDs

## - Description:

**Weak Session IDs occur when session identifiers are predictable or not sufficiently random, making them easy to guess or exploit.**

## - How it works:

**The application generates session IDs using weak patterns or predictable values, allowing attackers to guess or reuse valid session tokens.**

## - Steps:

1. **Observe session IDs assigned to users.**
2. **Analyze pattern or predictability of IDs.**
3. **Attempt to guess a valid session ID.**
4. **Use the guessed ID to impersonate a valid user session.**

## - Result:

**Session hijacking and unauthorized access to user accounts.**

## - Mitigation:

- **Generate strong, random session IDs**
- **Use HTTPS for secure transmission**
- **Regenerate session IDs after login and privilege changes**
- **Set proper session expiration and timeout**

# 10. DOM-Based XSS

## - Description:

**DOM-Based XSS is a vulnerability where malicious scripts are executed due to unsafe handling of user input in the browser's Document Object Model (DOM), without server-side processing.**

## - How it works:

**The application uses user-controlled input directly in JavaScript to modify the DOM, allowing attackers to inject and execute malicious scripts in the victim's browser.**

## - Steps:

1. **Identify a page that uses user input in client-side JavaScript.**
2. **Inject a malicious script payload into the input.**
3. **The script is reflected in the DOM without sanitization.**
4. **The browser executes the injected JavaScript code.**

## - Result:

**Execution of malicious scripts in the user's browser, leading to session theft, data leakage, or phishing attacks.**

## - Mitigation:

- **Properly sanitize and encode user input in JavaScript**
- **Avoid unsafe DOM functions like innerHTML**
- **Use safe APIs like textContent**
- **Implement Content Security Policy (CSP)**

# 11. Reflected XSS

## - Description:

**Reflected XSS occurs when a malicious script is injected and immediately reflected in the server response without proper sanitization.**

## - How it works:

**The application includes user input directly in the response, allowing injected JavaScript to be executed in the victim's browser.**

## - Steps:

1. **Inject payload:**

```
<script>alert(Hi)</script>
```

2. **Submit input through a vulnerable parameter.**
3. **Server reflects the input in the response.**
4. **Browser executes the script.**

## - Result:

**Execution of attacker-controlled script in the victim's browser.**

## - Mitigation:

- **Input/output encoding**
- **Input sanitization**
- **Use security headers like CSP**

# 12. Stored XSS

## - Description:

**Stored XSS occurs when malicious scripts are saved on the server and executed when users access the affected content.**

## - How it works:

**The attacker stores a script in the database or server storage, which is later served to users without sanitization.**

## - Steps:

1. **Inject malicious script into input field.**
2. **Script is stored in database.**
3. **Another user loads the page.**
4. **Script executes in their browser.**

## - Result:

**Persistent attack affecting multiple users.**

## - Mitigation:

- **Sanitize inputs**
- **Encode outputs**
- **Validate user-generated content**

# 13. CSP Bypass

## - Description:

**CSP Bypass occurs when attackers find ways to bypass Content Security Policy restrictions to execute malicious scripts.**

## - How it works:

**Weak or misconfigured CSP rules allow execution of unauthorized scripts or external resources.**

## - Steps:

1. **Identify weak CSP configuration.**
2. **Find allowed script sources or loopholes.**
3. **Inject or load malicious script through allowed sources.**
4. **Execute bypassed payload.**

## - Result:

**Execution of malicious scripts despite CSP protection.**

## - Mitigation:

- **Enforce strict CSP rules**
- **Disable inline scripts**
- **Avoid unsafe sources (*, unsafe-inline)**

# 14. JavaScript Vulnerabilities

## - Description:

**JavaScript vulnerabilities occur due to insecure client-side logic and lack of proper validation.**

## - How it works:

**Attackers manipulate client-side code or inputs because validation is not enforced on the server.**

## - Steps:

1. **Inspect client-side JavaScript.**
2. **Modify or bypass validation logic.**
3. **Send manipulated requests to server.**
4. **Server processes insecure input.**

## - Result:

**Bypass of security controls and unauthorized actions.**

## - Mitigation:

- **Always validate on server-side**
- **Do not trust client-side validation**
- **Secure JavaScript logic**

# 15. Authorization Bypass

## - Description:

**Authorization bypass occurs when users access restricted resources without proper permission checks.**

## - How it works:

**The application fails to properly enforce role or permission checks on sensitive resources.**

## - Steps:

1. **Identify restricted endpoint.**
2. **Modify request or URL.**
3. **Access resource without proper role.**
4. **Server fails to block access.**

## - Result:

**Unauthorized access to protected data or functions.**

## - Mitigation:

- **Implement proper access control**
- **Use role-based authorization (RBAC)**
- **Enforce server-side permission checks**

# 16. Open HTTP Redirect

## - Description:

**Open redirect occurs when an application redirects users to untrusted external URLs.**

## - How it works:

**User-controlled input is used directly in redirect logic without validation.**

## - Steps:

1. **Find redirect parameter.**
2. **Replace it with malicious URL.**
3. **Application redirects user.**
4. **User is sent to attacker site.**

## - Result:

**Phishing attacks and malicious redirections.**

## - Mitigation:

- **Validate redirect URLs**
- **Use allowlists for trusted domains**
- **Avoid user-controlled redirects**

# 17. Cryptography Issues

## - Description:

**Cryptography issues occur when weak or improper encryption methods are used to protect sensitive data.**

## - How it works:

**Weak algorithms or poor key management lead to easily breakable encryption.**

## - Steps:

1. **Identify weak encryption method.**
2. **Attempt to decrypt or analyze data.**
3. **Exploit weak keys or algorithms.**
4. **Access sensitive information.**

## - Result:

**Exposure of confidential data.**

## - Mitigation:

- **Use strong algorithms (AES, SHA-256)**
- **Secure key management**
- **Avoid deprecated encryption methods**

# 18. API Vulnerabilities

## - Description:

**API vulnerabilities occur due to weak authentication, authorization, or input validation in APIs.**

## - How it works:

**Attackers exploit insecure endpoints to access or manipulate data without proper authorization.**

## - Steps:

1. **Identify API endpoint.**
2. **Test for missing authentication or weak validation.**
3. **Send unauthorized requests.**
4. **API processes malicious input.**

## - Result:

**Data leakage or unauthorized actions via API.**

## - Mitigation:

- **Use authentication tokens (JWT, OAuth)**
- **Implement rate limiting**
- **Validate all inputs**
- **Enforce authorization checks**

---

# Results and Findings:

**During this project, multiple vulnerabilities were successfully exploited within DVWA. The most critical vulnerabilities identified include:**

- **SQL Injection (high impact on data confidentiality)**
- **Command Injection (full system compromise)**
- **File Upload (remote code execution)**
- **XSS (session hijacking risk)**

**These vulnerabilities demonstrate how insecure coding practices can lead to severe security breaches.**

---

# Mitigation and Recommendations:

**To secure web applications, the following practices should be implemented:**

- **Always validate and sanitize user inputs**
- **Use prepared statements for database queries**
- **Implement strong authentication mechanisms**
- **Apply least privilege principle**
- **Use HTTPS for secure communication**
- **Regularly test applications for vulnerabilities**
- **Keep systems updated**

---

# Conclusion:

**This project provided practical experience in identifying and exploiting common web application vulnerabilities using the Damn Vulnerable Web Application (DVWA). It helped demonstrate how attackers can take advantage of weaknesses such as SQL Injection, XSS, Command Injection, File Upload, and other security flaws to compromise web applications.**

**The exercises showed how improper input validation, weak authentication, and insecure configurations can lead to serious impacts such as data leakage, unauthorized access, and system compromise. Each vulnerability was analyzed along with its exploitation method and corresponding mitigation techniques.**

**Overall, the project emphasized the importance of secure coding practices and proper security mechanisms in web development. Understanding both how these attacks work and how to prevent them is essential for building secure and reliable applications.**

---